Phát hiện botnet IoT bằng autoencoder nhẹ

Đề tài DT043 · Bài giữa kỳ An toàn Bảo mật Hệ thống thông tin

Tên: Hoàng Thanh Trà

MSSV: 1150080120

Lớp: 11CNPM2

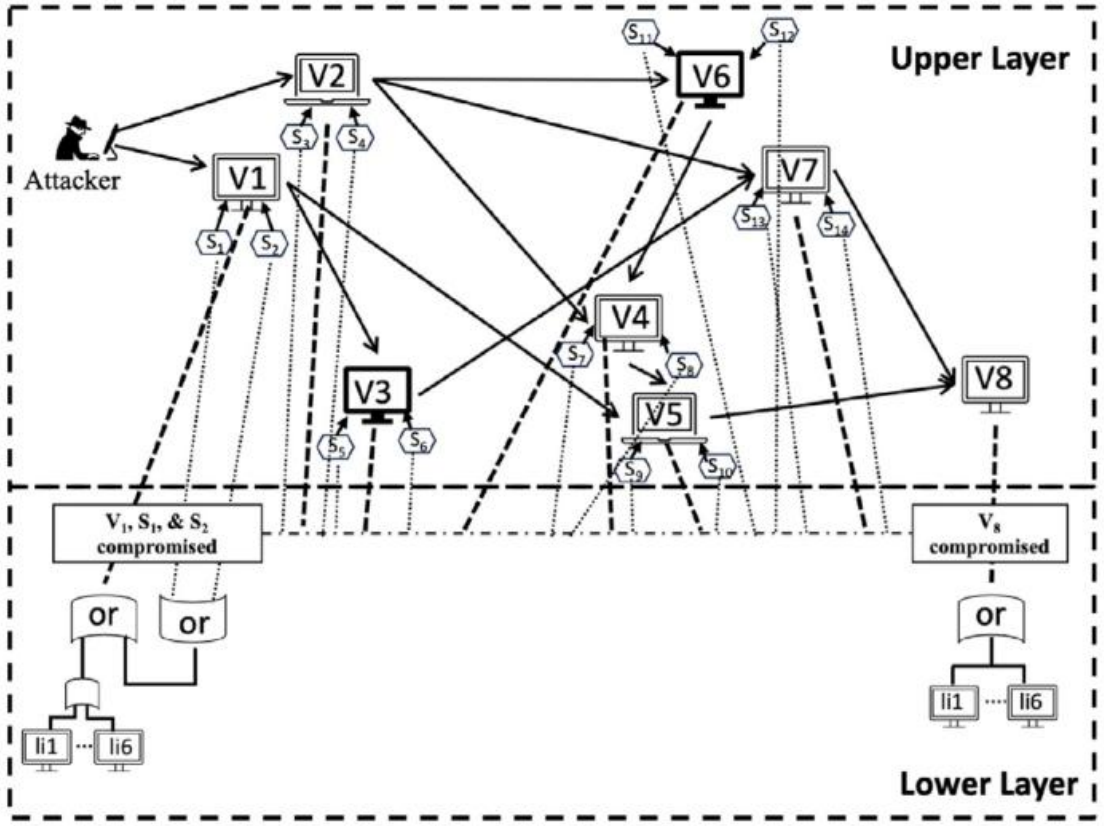
1. Phân tích bài báo · 2. Các bước làm bài · 3. Kết quả · 4. So sánh với bài báo · 5. Dự định làm tiếp

Bài toán mà bài báo đặt ra

Bối cảnh	Tấn công mạng nhắm vào hạ tầng và thiết bị IoT ngày càng nhiều. Moving Target Defence (MTD) liên tục thay đổi bề mặt tấn công – địa chỉ, hệ điều hành, số bản sao máy – để kẻ tấn công khó khai thác và tốn công hơn.
Vấn đề cốt lõi	Chưa có đánh giá hiệu quả khi kết hợp nhiều kỹ thuật MTD theo thời gian, chi phí và công sức tấn công. IoT không đồng nhất, tài nguyên hạn chế, nên MTD có thể tiêu tốn quá mức tài nguyên mạng và hệ thống.
Yêu cầu đặt ra	Cần phương pháp định lượng mức an toàn của từng trạng thái mạng sau mỗi lần MTD, so sánh shuffle, diversity, redundancy và tổ hợp của chúng để chọn chiến lược phòng thủ tốt nhất.

Masud, Keshk, Moustafa, Turnbull, Susilo – Vulnerability defence using hybrid moving target defence in Internet of Things systems. Computers & Security 153 (2025) 104380 · DOI 10.1016/j.cose.2025.104380 · Open Access (CC BY)

Phương pháp: mô hình 3-layer THARM và ba kỹ thuật MTD



Hình 3 của bài báo: tầng trên (máy ảo, dịch vụ) và tầng dưới (cây lỗ hổng OR)

Mô hình TH = (N, U, S, L, C)

- N: các trạng thái mạng theo thời gian
- U: đồ thị tấn công giữa 8 máy ảo
- S: đồ thị tấn công giữa 14 dịch vụ
- L: cây tấn công (cổng OR) gồm lỗ hổng OS + dịch vụ của từng máy ảo

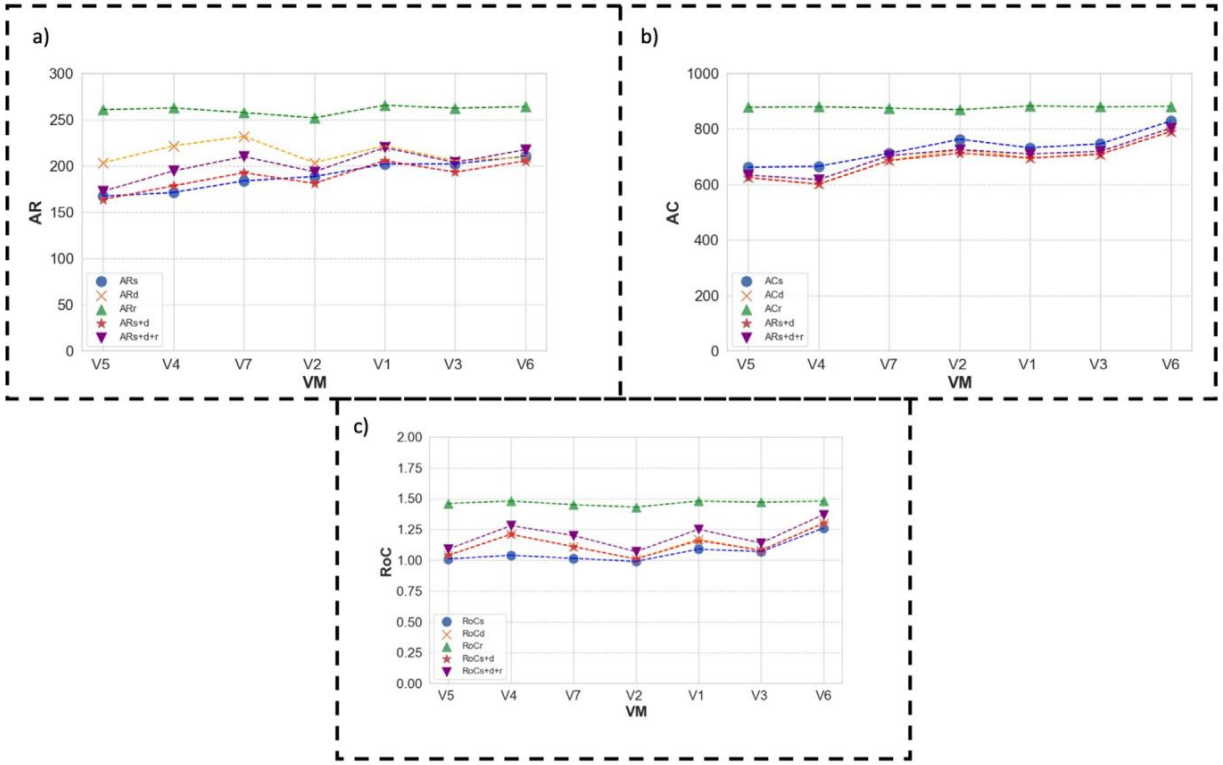
MTD kích hoạt theo thời gian (chu kỳ 10 phút)

- Shuffle: di trú máy ảo, đổi IP qua SDN
- Diversity: đổi hệ điều hành
- Redundancy: nhân bản máy ảo

Chỉ số đánh giá

- ASP (xác suất tấn công thành công), rủi ro AR, chi phí tấn công AC, RoC = R/AC
- Độ trung tâm của đồ thị để chọn máy ảo cần bảo vệ

Thực nghiệm, kết quả và hạn chế của bài báo



Hình 6 của bài báo: AR, AC, RoC sau khi áp MTD (s, d, r, s+d, s+d+r) lên từng máy ảo

Thực nghiệm

Mô phỏng 4 host, 8 máy ảo (Linux, Windows 11, FreeBSD), 18 CVE hệ điều hành + 14 CVE dịch vụ kèm CVSS; không dùng dataset.

Kết quả

Shuffle + diversity cho rủi ro thấp nhất, chi phí tấn công cao hơn. Ví dụ V5: R_s 167.38 · R_d 163.73 · R_r 260.84.

Hạn chế

Chỉ mô phỏng; xác suất khai thác tĩnh; chỉ MTD theo thời gian, không có bộ phát hiện kích hoạt; không công bố mã.

→ Đề tài: Track A phát hiện botnet bằng autoencoder (theo Meidan 2018) · Track B tái lập THARM và MTD lai do cảnh báo AE kích hoạt

Bước 1–2: Đọc bài báo, lập kế hoạch

Track A – Phát hiện

Theo Meidan 2018: AE per-device trên N-BaIoT, thu nhỏ, int8, chạy trên edge (RQ1–RQ5)

Track B – MTD

Theo Masud 2025: tái lập THARM bằng đúng dữ liệu bài, cảnh báo AE kích hoạt MTD lai (RQ6, E11)

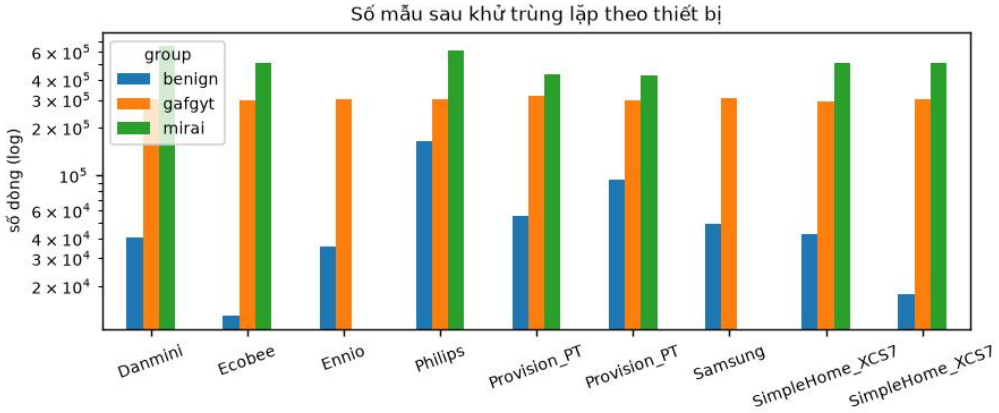
Làm gì

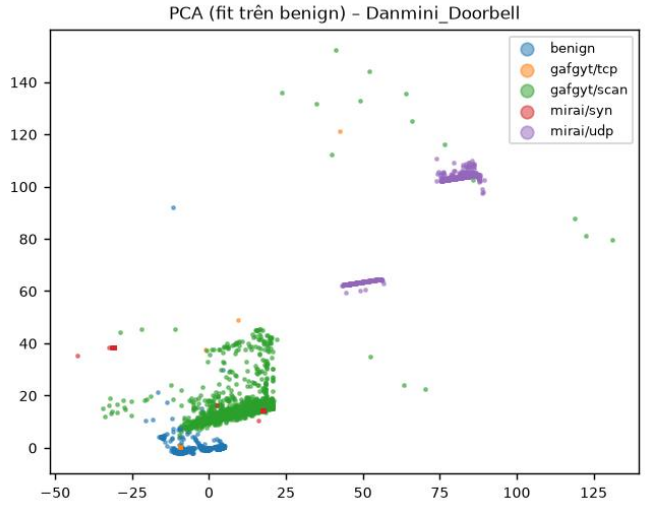
- Đọc Masud 2025 (MTD, mô hình THARM) và Meidan 2018 (autoencoder trên N-BaIoT)
- Viết phiếu tái lập: liệt kê ≥ 5 hạn chế của bài, đặt câu hỏi nghiên cứu

Kết quả

- Bài chính không có bộ phát hiện → chia 2 track: A phát hiện bằng AE, B tái lập THARM + MTD lai
- 6 câu hỏi nghiên cứu RQ1–RQ6, ma trận 11 thí nghiệm E1–E11

Bước 3–4: Tải dữ liệu, làm sạch, EDA





Hình chạy trong notebook src/DT043_walkthrough.ipynb (mục 1–2)

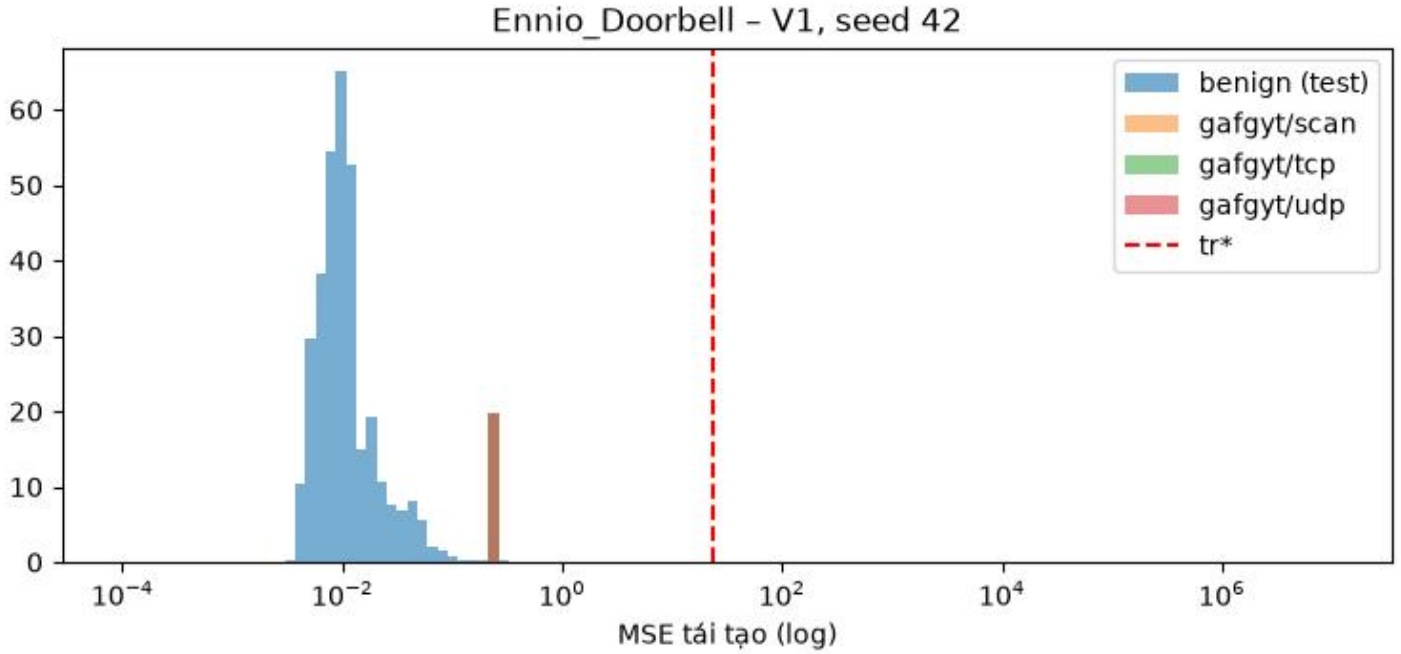
Làm gì

- Tải N-BaIoT 1.77 GB, kiểm SHA-256 từng file
- Kiểm schema 115 cột, khử trùng lặp, đo mất cân bằng, PCA

Kết quả

- 107 file khớp SHA-256 · 7.06 triệu dòng · 9 thiết bị
- Loại 157,779 dòng trùng; attack nhiều hơn benign 5.5–61.8× → dùng macro-F1, FPR

Bước 5: Chia chống rò rỉ, huấn luyện AE, chọn ngưỡng



Hình chạy trong notebook src/DT043_walkthrough.ipynb (mục 3–4)

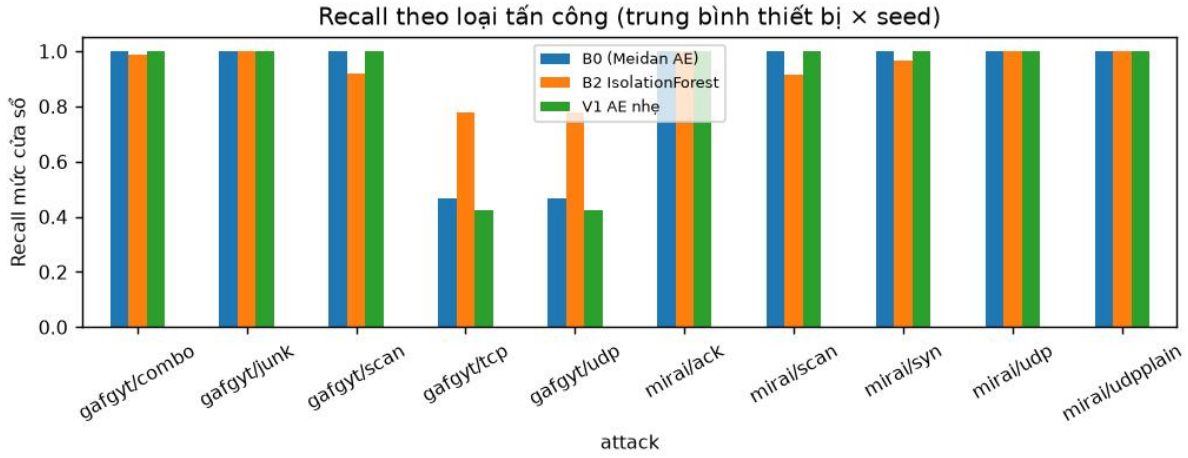
Làm gì

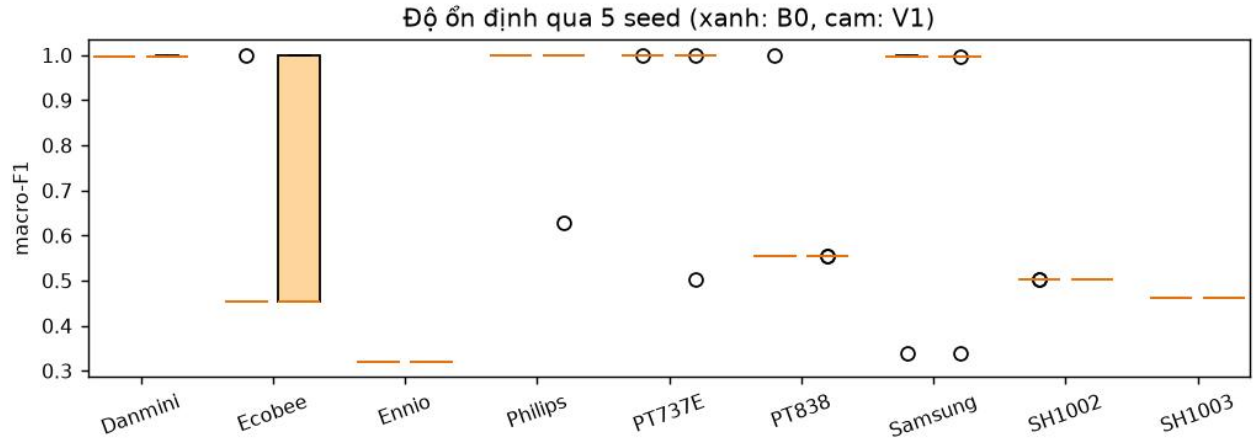
- Chia benign theo thời gian 1/3 · 1/3 · 1/3, đệm 500 dòng; scaler fit trên train
- AE V1 115-32-8-32-115 học benign; ngưỡng tr* = mean + std của MSE

Kết quả

- Benign nằm dưới ngưỡng – không báo động giả; 30 test chống rò rỉ
- Flood gafgyt tcp/udp cũng nằm dưới tr* → nguyên nhân bỏ sót chính

Bước 6–7: Chốt tiêu chí, chạy 468 lần





Hình chạy trong notebook src/DT043_walkthrough.ipynb (mục 7)

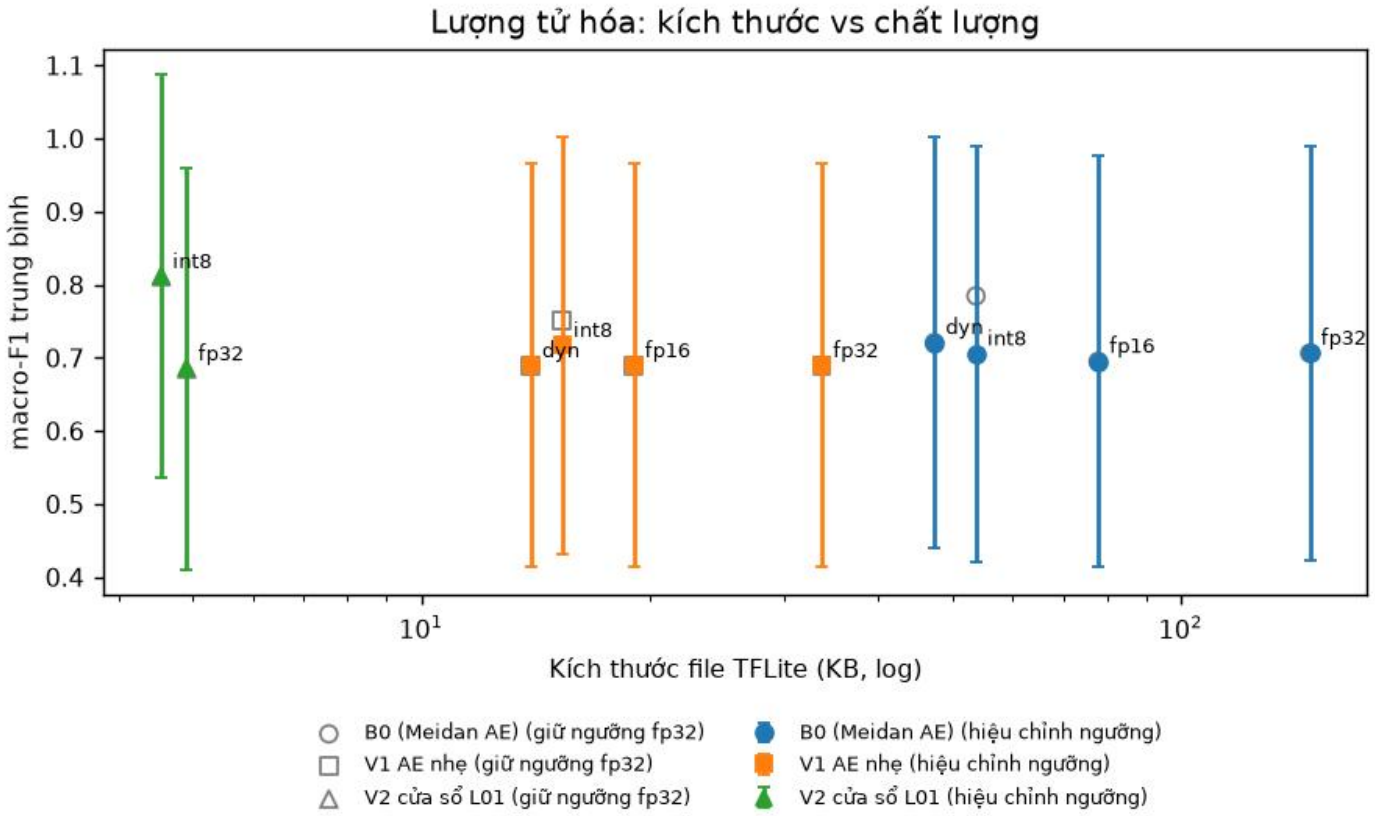
Làm gì

- Chốt tiêu chí thành công, lưu SHA-256 trước khi chạy
- Chạy 468 lần: cấu hình × 9 thiết bị × 5 seed

Kết quả

- B0: FPR 0.0025, TPR 0.822; 8/10 loại tấn công recall ≈ 1, chỉ tcp/udp thấp
- Kết quả dao động theo seed → phải chạy nhiều lần

Bước 7: Lượng tử hóa int8, đo trên edge



Hình chạy trong notebook src/DT043_walkthrough.ipynb (mục 5–7)

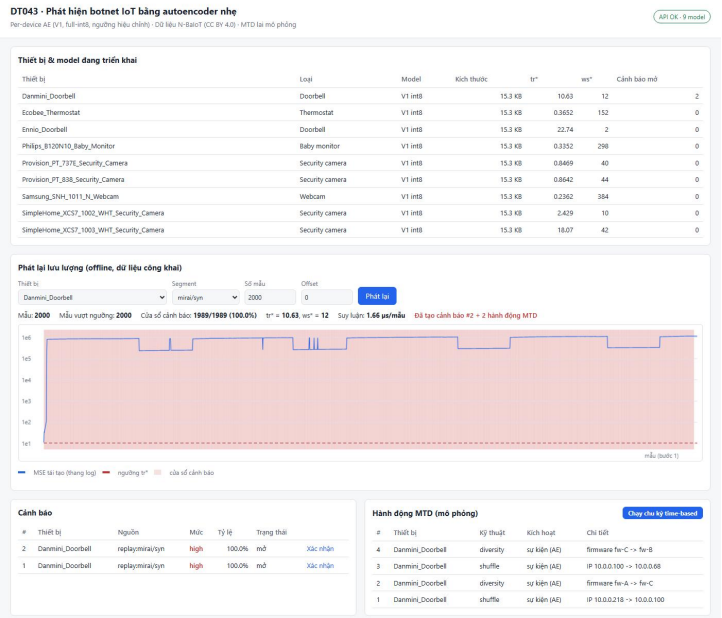
Làm gì

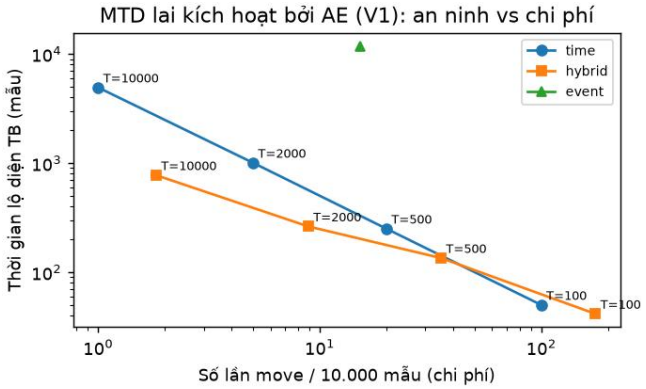
- Chuyển TFLite fp32 / fp16 / dynamic / int8, hiệu chỉnh lại ngưỡng
- Đo trên 1 lõi E-core: độ trễ, RAM, thông lượng

Kết quả

- V1 int8 15.3 KB (45.5% bản fp32), macro-F1 giảm 0.028 – RQ4 không đạt
- p95 10.8 µs/mẫu · ~103 nghìn mẫu/s · RAM +4.6 MB

Bước 8–9: Dựng hệ thống 4 tầng, MTD lai





Làm gì

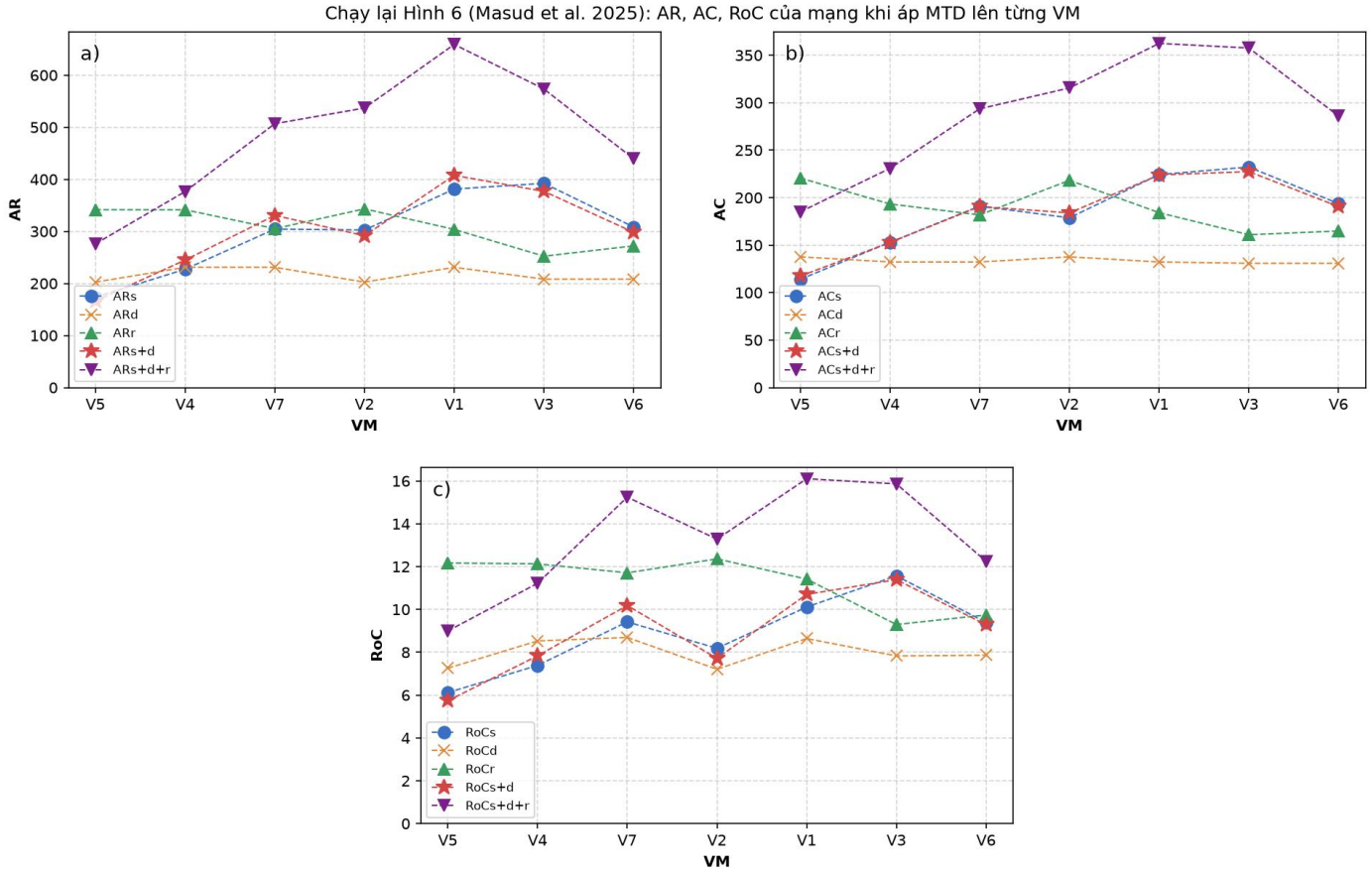
- AI xuất gói có SHA-256 → FastAPI → SQLite → dashboard
- Mô phỏng MTD theo thời gian, theo cảnh báo và lai

Kết quả

- mirai/syn: 1989/1989 cửa sổ cảnh báo → tự kích hoạt 2 hành động MTD
- MTD lai: cùng mức lộ diện, số lần đổi ít hơn ≈ 55%

Hình chạy trong notebook src/DT043_walkthrough.ipynb (mục 8, 10)

Bước 10: Tái lập bài báo chính (E11, E12)



Hình chạy trong notebook src/DT043_walkthrough.ipynb (mục 9)

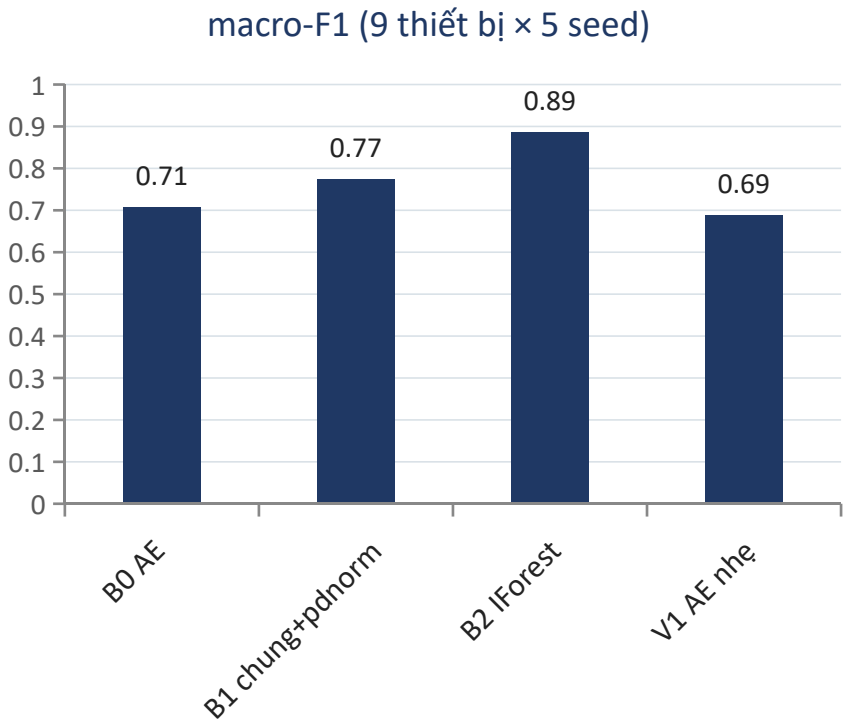
Làm gì

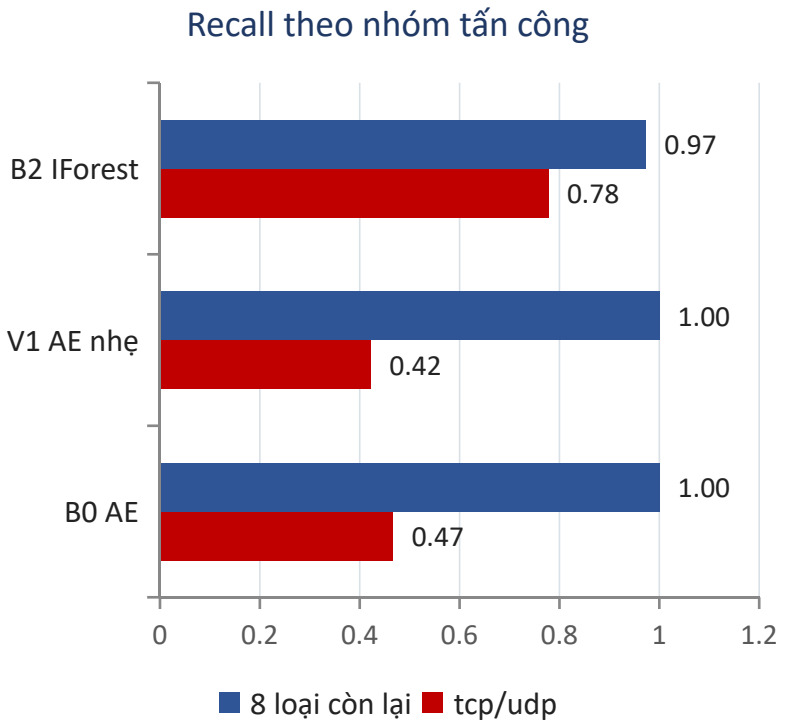
- Dựng 3-layer THARM bằng đúng 32 CVE, 8 VM, topology của bài
- Vẽ lại Hình 6, 7; dò công thức AC, ASP mà bài không in rõ

Kết quả

- Rủi ro VM khớp ±0.01, đường tấn công −0.8%
- Tổng AC lệch ≤ 3.4%, RoC ≤ 8.4% với công thức suy ra; tổng R của D, R vẫn lệch 13–17%
- Hình 6(a): 11/35 điểm lệch ≤ 5%

Kết quả: FPR rất thấp, lỗi tập trung ở flood tcp/udp





0.0025

FPR của B0 (Meidan: 0.007)

0.822

TPR của B0 (Meidan: 100%)

Nguồn bị giả mạo liên tục → mỗi gói như "luồng mới" (weight = 1, variance = 0), MSE nằm dưới ngưỡng mean+std. Tám loại tấn công còn lại recall ≈ 1.0.

Đối chiếu 6 câu hỏi nghiên cứu với tiêu chí chốt trước

RQ	Câu hỏi	Giá trị đo	Kết luận
RQ1	Tái lập baseline Meidan 2018	TPR=0.8222, FPR=0.0025	Không đạt
RQ2	Per-device hơn model chung?	thắng 4/9, p=0.2852	Không đạt
RQ3	AE nhẹ giữ chất lượng?	tham số 21.9%, Δ=+0.0171	Không đạt
RQ4	int8 + hiệu chỉnh ngưỡng	Δ=-0.0278, kích thước 45.5%	Không đạt
RQ5	Chạy được trên edge?	p95=10.8 µs, 15.3 KB, ΔRSS=4.6 MB	Đạt
RQ6	AE kích hoạt MTD lai	lộ diện 263 vs 251 mẫu; đổi 8.9 vs 20.0 lần/10k (so với time-based)	Ủng hộ H6

Tiêu chí chốt và lưu SHA-256 trước khi chạy, không chỉnh sau khi có kết quả. RQ1 hụt vì flood tcp/udp; RQ2–RQ4 sát ngưỡng; RQ5 đạt với biên rất lớn; MTD lai đổi ít hơn ≈ 55% ở cùng mức lộ diện.

So sánh với Meidan 2018: FPR khớp, TPR thấp hơn

Tiêu chí	Meidan 2018	DT043
FPR	0.007 ± 0.01	0.0025 (thấp hơn, chưa kiểm định)
TPR	100%	0.822
Chia dữ liệu	1 lần, không khử trùng lặp	theo thời gian, đệm 500 dòng, khử 157,779 dòng trùng
Số lần chạy	1	468 lần, 5 seed
Thời gian phát hiện	174 ± 212 ms	theo số mẫu (CSV không có timestamp)
Chạy trên edge	không đo	int8 15.3 KB, 10.8 µs/mẫu

Vì sao TPR thấp hơn?

1. Phần bỏ sót nằm ở flood tcp/udp BASHLITE giả mạo nguồn – mỗi gói như luồng mới, giống benign; 8 loại còn lại recall ≈ 1.0
2. Ngưỡng mean + std bị đuôi MSE kéo cao; bài cũng tối ưu η, epochs riêng từng thiết bị
3. Rò rỉ không giải thích được: chia ngẫu nhiên (E6) có TPR 0.804, FPR 0.0006. Chưa tách được nguyên nhân duy nhất

Phương pháp giữ đúng Meidan: AE 75-50-33-25% số chiều, tr = mean + std trên DSopt, vote cửa sổ ws*; khác biệt có chủ đích ghi trong phiếu tái lập.*

So sánh với Masud 2025: tái lập bằng đúng dữ liệu bài

Rủi ro	Paper	Tái lập
R_V5 (Windows 11)	12.58	12.578
R_V7 · R_V8	8.67 · 5.62	8.667 · 5.625
Đường shuffle	43.15	42.81 (-0.8%)
Đường diversity	43.31	42.97 (-0.8%)
Đường redundancy	43.31	42.97 (-0.8%)

HARM rút gọn (chu kỳ 20): ASP không MTD 0.498 → shuffle 0.409 · diversity 0.437 · kết hợp 0.327. MTD lai dùng cảnh báo AE: cùng chi phí, lộ diện ít hơn time-based.

6/7

kết luận tái lập được (topology Hình 3)

3/7

nếu dùng đúng Definition 2

Khớp: rủi ro VM ±0.01, đường −0.8%. Gần khớp (E12, công thức AC suy ra): tổng AC ≤ 3.4%, RoC ≤ 8.4%. Chưa khớp: tổng R của D, R (−13%, −17%), Hình 5(b), 6, 7, 9. "S+D tốt nhất" chỉ đạt ở ví dụ V5, topology Hình 3.

Dữ liệu paper: 18 CVE OS + 14 CVE dịch vụ (Bảng 1–2), 8 VM, Definition 2; shuffle/diversity/redundancy theo mục 4.2–4.4

Kết luận và dự định làm tiếp

Kết luận

- AE per-device bắt gần tuyệt đối 8/10 loại tấn công, FPR 0.0025; bỏ sót flood tcp/udp giả mạo nguồn
- V1 int8 15.3 KB, ~11 µs/mẫu trên 1 lõi – khả thi cho edge
- MTD lai do AE kích hoạt: cùng mức lộ diện, đổi ít hơn ≈ 55%
- Tái lập Masud: khớp mức VM/đường, tổng AC và RoC gần khớp; tổng R, Hình 5–9 chưa khớp vì bài thiếu thông tin

Dự định làm tiếp

Bắt flood giả mạo nguồn

đặc trưng tốc độ luồng mới, entropy IP nguồn; luật chạy song song AE

Chạy trên Raspberry Pi

đo độ trễ và điện năng thật, tính cả bước trích đặc trưng

MTD trên mạng giả lập

Mininet / SDN, đo gián đoạn dịch vụ khi đổi IP, firmware

Thử dữ liệu mới

CICIoT2023, IoT-23 – kiểm tra khả năng tổng quát

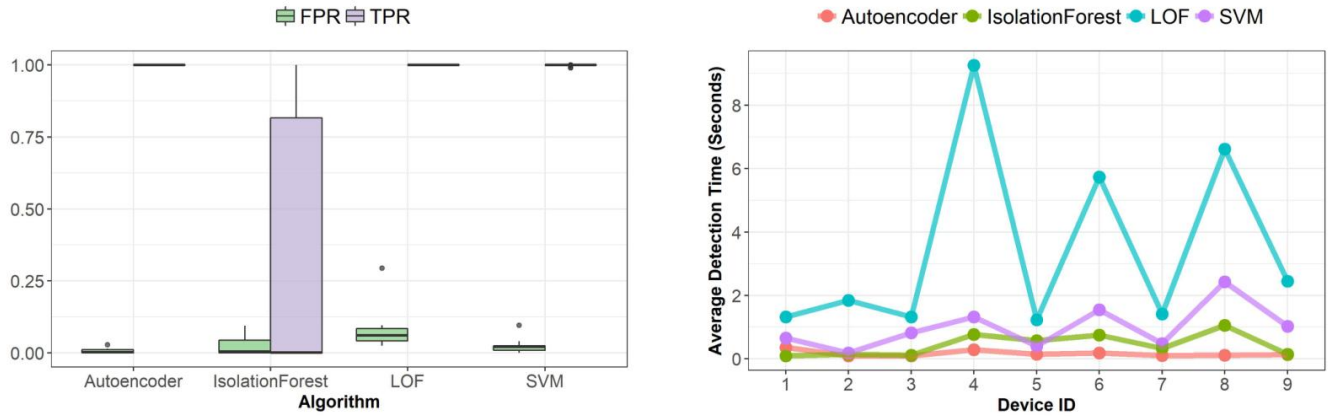
Hoàn thiện tái lập

xin tác giả công thức AC, chuỗi 8 đường Hình 7, mô hình Hình 8–9

Mã nguồn: github.com/thtra04/LAB_AT_BMHTTT (thư mục DT043) · Em cảm ơn thầy và các bạn!

N-BaIoT (Meidan et al., 2018): bài toán, phương pháp, kết quả

Bài toán	Phát hiện thiết bị IoT đã bị nhiễm botnet ngay khi bắt đầu tấn công, chỉ từ lưu lượng mạng.
Phương pháp	Lây nhiễm Mirai, BASHLITE vào 9 thiết bị thật; 115 đặc trưng thống kê; mỗi thiết bị một deep AE học benign; tr* = mean + std, majority vote trên cửa sổ ws*.
Kết quả	TPR 100%, FPR 0.007 ± 0.01 – tốt hơn Isolation Forest, LOF, SVM; phát hiện trong 174 ± 212 ms.
Hạn chế	Chia dữ liệu một lần, không khoảng tin cậy; ngưỡng/cửa sổ chọn để FPR = 0; không lượng tử hóa, không đo trên edge.



(a) Methods' detection accuracy (b) Methods' detection time (seconds)

Hình 2 (a, b) của bài báo: TPR/FPR và thời gian phát hiện của autoencoder so với Isolation Forest, LOF, SVM

Phương pháp phát hiện: chi tiết từng bước

1. Chia benign của mỗi thiết bị theo thời gian: DStrn | DSopt | test (mỗi phần 1/3), bỏ 500 dòng đệm ở mỗi ranh giới; dữ liệu tấn công chỉ ở test.
2. Chuẩn hóa z-score, fit chỉ trên DStrn của đúng thiết bị (scaler gắn tên thiết bị).
3. Huấn luyện AE chỉ trên benign: Adam lr 1e-3, batch 256, early stopping (patience 8) theo DSopt, tối đa 100 epoch.
4. Điểm bất thường của mỗi mẫu = MSE tái tạo (tính bằng float64 vì có đặc trưng tới 5.9e17).
5. Ngưỡng tr* và cửa sổ ws* chỉ lấy từ DSopt benign (công thức bên phải).
6. Lượng tử hóa TFLite (fp16, dynamic, full-int8 với dữ liệu đại diện từ DStrn), rồi tính lại tr*, ws* bằng chính model int8.
7. Đánh giá trên test ở mức cửa sổ: macro-F1, FPR, TPR; 5 seed, KTC95, Wilcoxon + Holm trên 9 thiết bị.

Công thức

$MSE(x) = (1/115) \cdot \Sigma (x_i - \hat{x}_i)^2$

tr* = mean(MSE_DSopt) + std(MSE_DSopt)

Cửa sổ ws* mẫu bị cảnh báo khi hơn một nửa số mẫu có MSE > tr*

ws* = cửa sổ nhỏ nhất cho FPR = 0 trên DSopt

Vì sao hiệu chỉnh sau int8?
Lượng tử hóa làm lệch phân phối MSE; ngưỡng chỉ dùng DSopt benign nên tính lại được mà không chạm vào test.

Thực nghiệm: ma trận E1–E11 và môi trường chạy

ID	Nội dung	Số lần chạy	Phục vụ
E1	B0 – tái lập baseline Meidan	9 × 5	RQ1
E2	B1 global; B1 chung + chuẩn hóa riêng	9 × 5 × 2	RQ2
E3	B2 Isolation Forest	9 × 5	baseline
E4	V1 nhẹ; V2 × 5 cửa sổ	45 + 135	RQ3
E5	fp32 / fp16 / dyn / int8 × giữ / hiệu chỉnh ngưỡng	trong E1, E4	RQ4
E6	Đối chứng rò rỉ (cố tình chia ngẫu nhiên)	9 × 3	chống rò rỉ
E7	Ablation 4 quy tắc ngưỡng × 3 cửa sổ	trong E1, E4	ablation
E8	Edge benchmark P0 / P1	3 × 4 × 3 × 5 lượt	RQ5
E9	Robustness: né tránh, nhiễu, đầu độc 1% / 5%	9 × 3 × 3	an toàn
E10	MTD: HARM rút gọn + MTD lai do AE kích hoạt	5 seed Monte Carlo	RQ6
E11	Tái lập 3-layer THARM bằng dữ liệu bài báo	tất định, 24 test	bài chính

Môi trường

- i7-13650HX (6 P + 8 E core), RAM 23.7 GB, Windows 11
- Python 3.11.9, TensorFlow 2.21, scikit-learn 1.9
- Seed 42, 1337, 2024, 7, 123

Nguyên tắc

- Mỗi cấu hình là 1 file YAML; lưu config, seed, log, metric từng lần chạy
- Giữ cả lần chạy lỗi; tổng 468 kết quả, không chọn seed đẹp
- Test chỉ dùng để đánh giá cuối cùng

Tái lập 3-layer THARM: lấy gì từ bài báo, tính thế nào

Lấy từ bài báo	Nguồn
18 CVE hệ điều hành: CVSS, exploitability E, impact I, cost	Bảng 1
14 CVE của dịch vụ s1–s14	Bảng 2
Máy ảo → hệ điều hành (Linux, Windows 11, FreeBSD)	Hình 1
Máy ảo → dịch vụ; V8 là CSDL đích	Hình 3
Đồ thị tấn công giữa máy ảo (11 cạnh)	Definition 2
Shuffle: V5 di trú host 2 → host 4	Mục 4.2
Diversity: đổi hệ điều hành dự phòng	Mục 4.3
Redundancy: nhân bản V5 → V5’	Mục 4.4

1 **Dựng 3 tầng**

Cây tấn công OR cho từng máy ảo → đồ thị dịch vụ → đồ thị máy ảo, không tự đặt tham số.

2 **Rủi ro máy ảo**

$R_v = \Sigma E \cdot I$. Bản in ghi tích ($\prod$) nhưng chỉ dạng tổng mới khớp các số bài báo dùng.

3 **Đường tấn công**

Rủi ro đường = tổng R các máy ảo trên đường; AC = Σ cost; ASP theo OR độc lập.

4 **So với bài báo**

Áp shuffle / diversity / redundancy và tổ hợp; so số công bố và 7 kết luận định tính trên 2 topology.

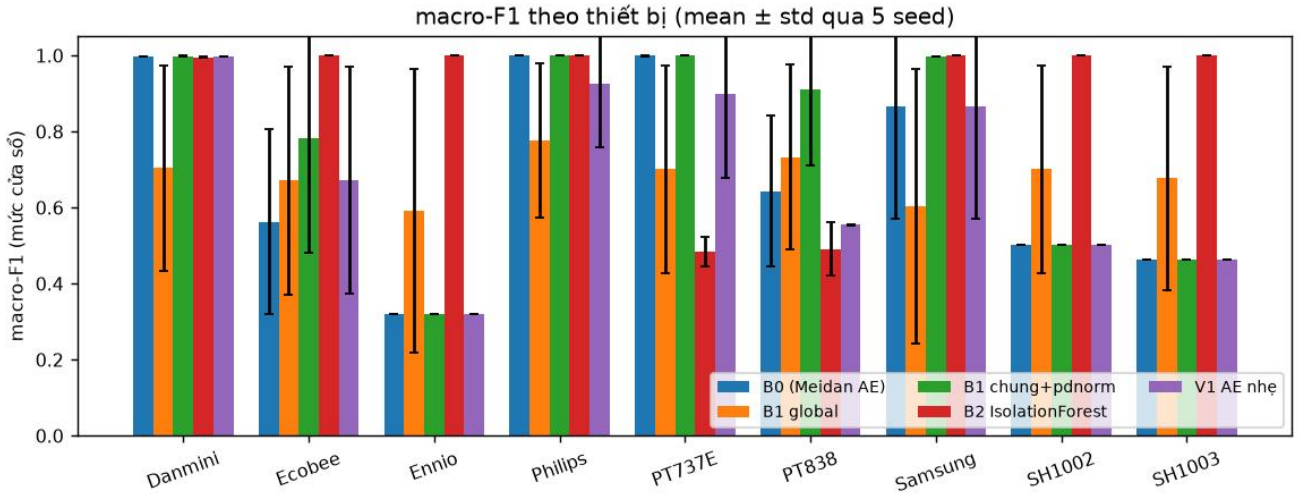
Mã: ai/src/nbaiot/mtd/tharm.py · script ai/scripts/10_tharm_paper_replication.py · 24 test khóa các số của bài báo · chi tiết: tailieu/docs/tai_lap_paper_chinh.md

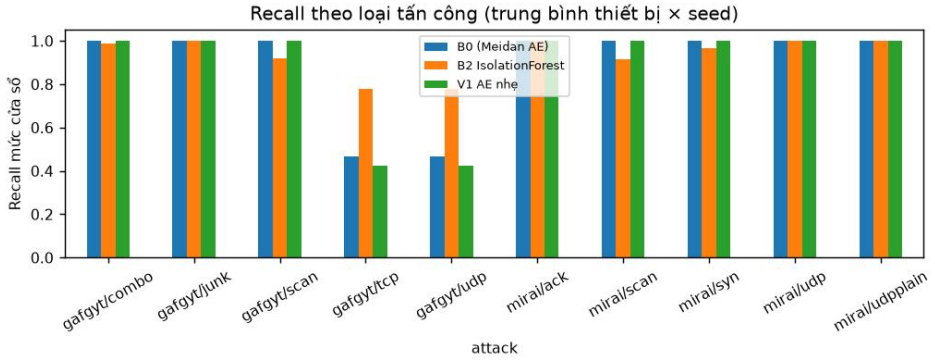
Kết quả tái lập Meidan 2018 trên N-BaIoT

Chỉ số	Meidan 2018	DT043 B0	V1 int8	Nhận xét
TPR	100%	0.822	0.829	Không đạt
FPR	0.007	0.0025	0.0016	Thấp hơn
ws* (mẫu)	20–82	152	113	Dài hơn
Model	172 kB	147.7 KB	15.3 KB	Nhỏ ~10×
Chia dữ liệu	1 lần	5 seed	5 seed	Chặt hơn

Vì sao lệch so với bài báo

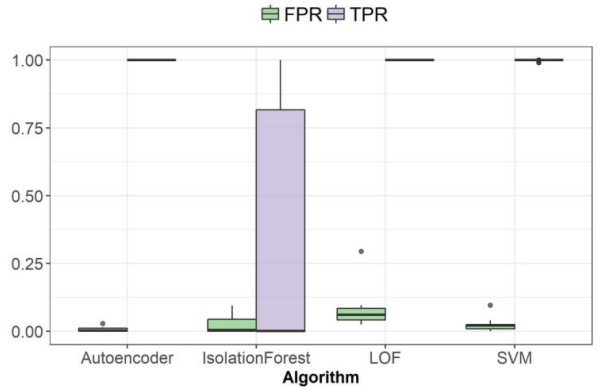
- Flood tcp/udp của BASHLITE giả mạo nguồn → giống benign, MSE dưới ngưỡng
- Đuôi MSE trên DSopt kéo ngưỡng mean + std lên cao
- Khử trùng lặp + vùng đệm → ws* dài hơn
- 8/10 loại tấn công còn lại recall ≈ 1



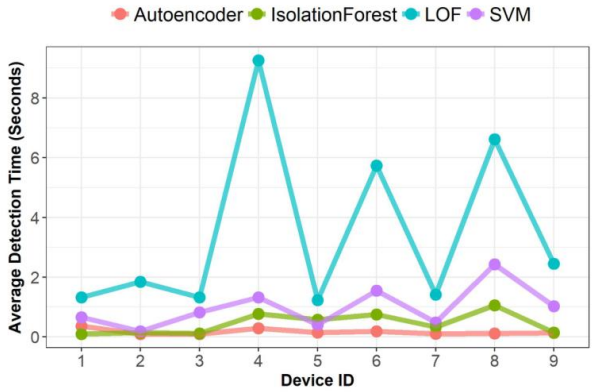


Chạy lại Hình 2 (a, b) của Meidan 2018

Hình 2 – bài báo

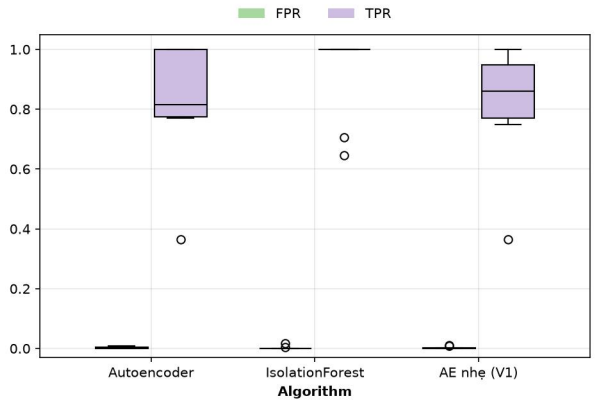


(a) Methods′ detection accuracy

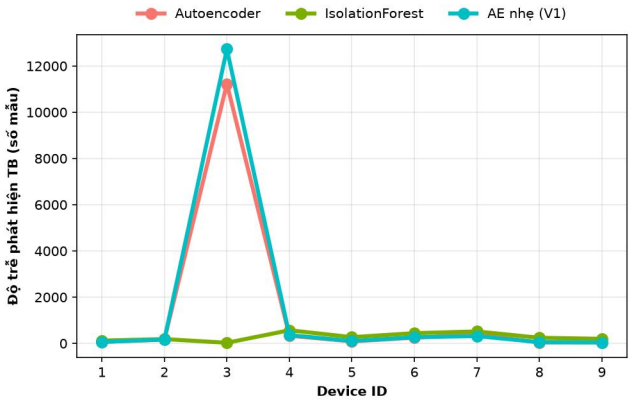


(b) Methods′ detection time (seconds)

Chạy lại – DT043 (468 lần chạy, trung bình theo seed)



(a) Độ chính xác phát hiện (9 thiết bị)



(b) Độ trễ phát hiện theo thiết bị (đo bằng số mẫu)

(a) Độ chính xác

- FPR gần 0 ở mọi thuật toán – khớp bài báo
- TPR của AE: bài 100%, chạy lại trung vị 0.82 do flood tcp/udp giả mạo nguồn
- Isolation Forest chạy lại tốt hơn AE (bài: ngược lại)

(b) Thời gian phát hiện

- Bài đo giây trên lưu lượng thật; N-BaIoT không có timestamp nên đo bằng số mẫu
- Thiết bị 3 (Ennio) trễ lớn vì AE bỏ sót tcp/udp

Chưa chạy lại được

- LOF, One-class SVM (chưa nằm trong ma trận thí nghiệm)
- Hình 2 (c, d): cần lưu lượng theo giờ – không có trong dataset

Kết quả tái lập bài báo chính (Masud 2025)

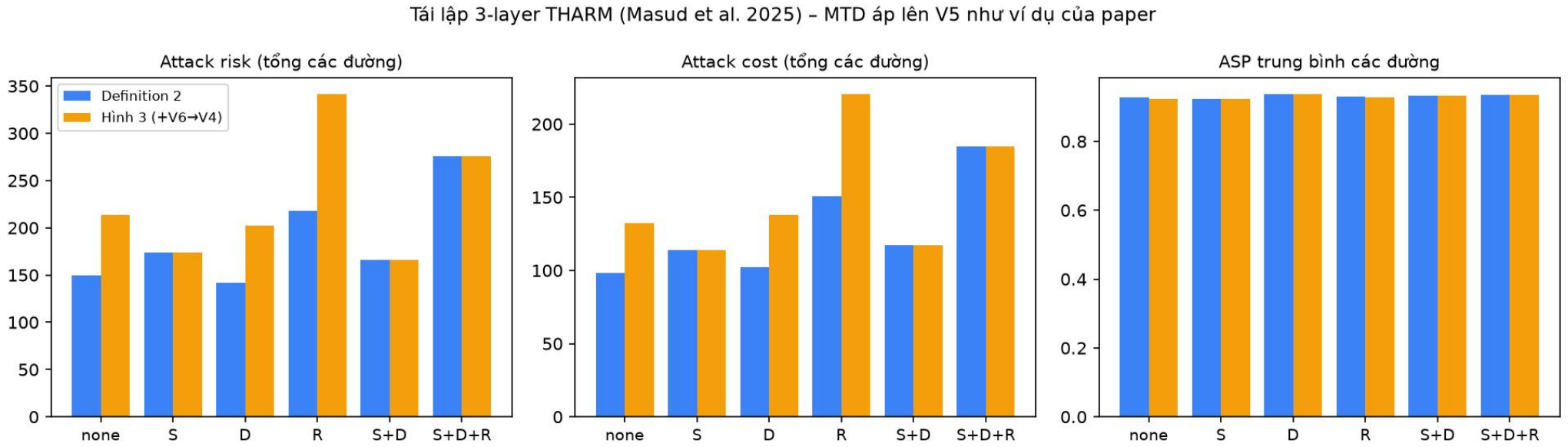
Mức so sánh	Bài báo	Tái lập	Kết luận
Rủi ro V4 · V5 · V7 · V8	12.47 · 12.58 · 8.67 · 5.62	12.47 · 12.58 · 8.67 · 5.63	Khớp ±0.01
3 đường tấn công ví dụ	43.15 · 43.31 · 43.31	42.81 · 42.97 · 42.97	Khớp −0.8%
Tổng mạng R_s · R_d · R_r	167 · 164 · 261	174 · 142 · 218	Không khớp

6/7

kết luận khớp (Hình 3)

3/7

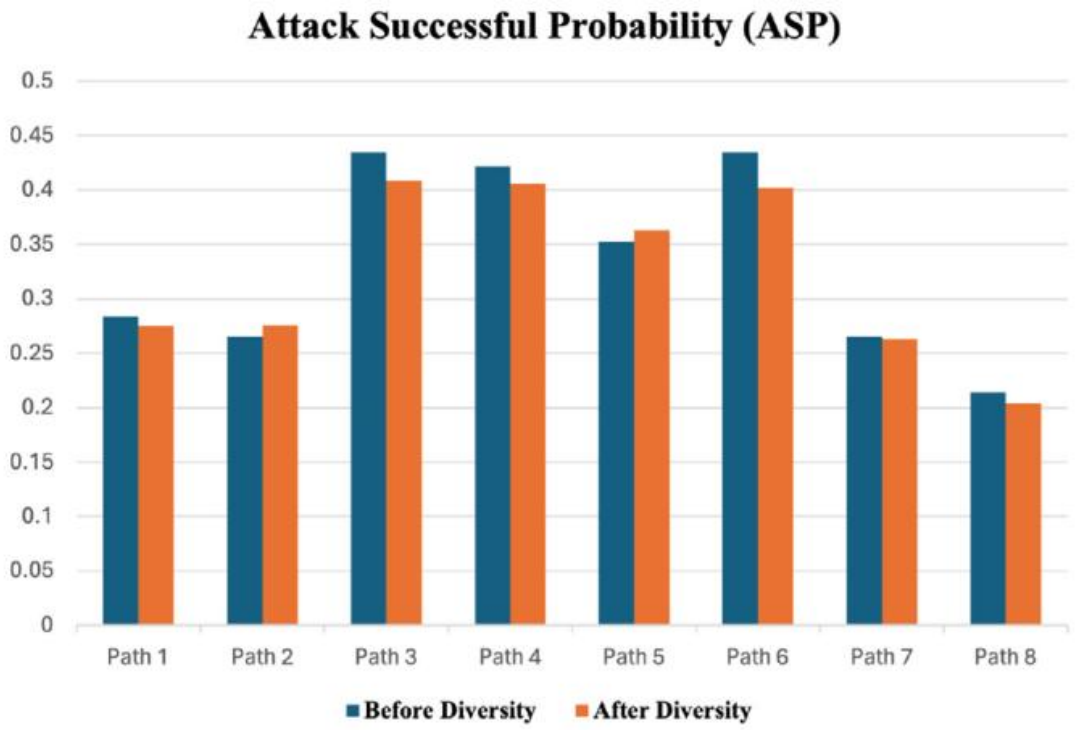
theo Definition 2



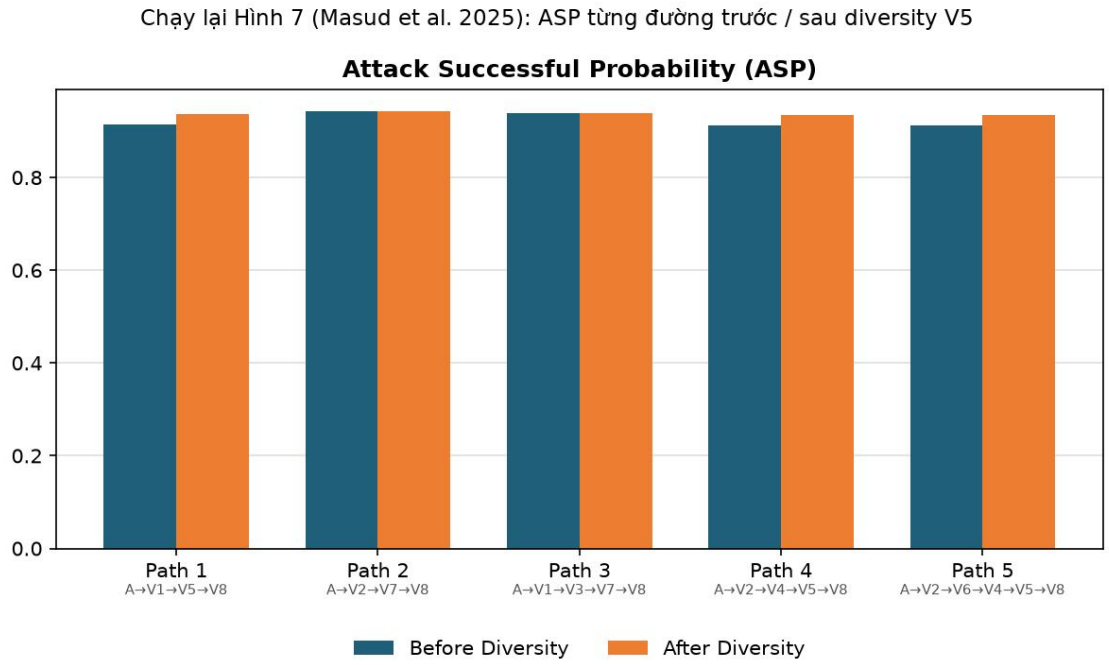
Tổng R không khớp với cả hai topology; AC, RoC xem E12. Kết luận "S + D tốt nhất" phụ thuộc cạnh V6 → V4 và chỉ đạt ở ví dụ V5.

Chạy lại Hình 7: ASP trước / sau diversity

Hình 7 – bài báo (8 đường)



Chạy lại – DT043 (5 đường tới V8, diversity V5)



Không tái lập được từng cột: topology chỉ có 5 đường tới V8 (bài vẽ 8, không in chuỗi VM). Hình phải dùng OR độc lập (0.91–0.94); Eq. 1 nguyên văn (E12) cho 0.24–0.48, cùng thang với bài (0.2–0.43). Đổi V5 Windows 11 → Linux làm ASP tăng ở cả hai cách, ngược lời bình của bài.

E12: công thức AC suy ra đưa tổng AC, RoC về gần bài

MTD trên V5	R bài	R E12	AC bài	AC E11	AC E12	RoC bài	RoC E12
Shuffle	167.4	173.6 (+3.7%)	666.2	113.9	654.3 (-1.8%)	1.01	1.07 (+6.3%)
Diversity	163.7	142.3 (-13.1%)	624.7	102.0	603.5 (-3.4%)	1.04	0.95 (-8.4%)
Redundancy	260.8	217.6 (-16.6%)	878.3	151.0	897.4 (+2.2%)	1.46	1.46 (-0.1%)

Cách tính suy ra

- AC theo VM = Σ CVSS BS × attack cost: khớp V5, V7, V8; V2 lệch −33.9%
- Là giả thuyết, dò trên 4 số bài in; tổng AC không dùng khi chọn công thức
- Eq. 1 nguyên văn: ASP đường 0.24–0.48, cùng thang Hình 7 (0.2–0.43)
- Tổng mạng lệch ≤ 5%: 5/9 (E11: 1/9)

Vẫn chưa khớp

- Tổng R: diversity −13.1%, redundancy −16.6%
- Hình 5(b), Hình 6 cùng thang nhưng khác hình dạng; Hình 7 không có chuỗi VM
- Hình 8: chỉ so tỷ lệ, mạng 400 VM là giả định; Hình 9: không có mô hình thời gian
- Bài không nhất quán: R_d = 163.73 nhưng Hình 6 vẽ ≈ 203

Nguyên mẫu: luồng dữ liệu, gói triển khai và API



Gói triển khai mỗi thiết bị (artifacts/deploy/<thiết bị>/)

- model.tflite – V1 int8, 15,688 byte
- scaler.json – mean/std 115 cột, gắn tên thiết bị
- meta.json – tr*, ws*, danh sách cột, SHA-256 của model
- Backend từ chối nạp nếu checksum lệch hoặc sai thiết bị

Cơ sở dữ liệu (6 bảng)

- devices, models, detection_batches, alerts, mtd_actions, experiment_results – không lưu dữ liệu thô

API	Chức năng
GET /api/health, /api/devices	trạng thái, 9 model đã nạp
POST /api/detect	suy luận một lô đặc trưng
POST /api/replay/{device}	phát lại dữ liệu test công khai
GET /api/alerts · POST …/ack	xem, xác nhận cảnh báo
GET /api/mtd/actions · POST /api/mtd/tick	nhật ký MTD, MTD theo thời gian
GET /api/experiments	bảng kết quả thực nghiệm

66 test tự động: 30 AI + 24 tái lập THARM + 12 backend/CSDL (gói bị sửa 1 byte, path traversal, sai số chiều, lô quá lớn)